

Exposé für die Bachelorarbeit

Titel der Bachelorarbeit:	Graph-basierte Systemarchitektur vs. monolithische Systemprompt-Struktur: Eine empirische Evaluation von Halluzinationsrate, Nachvollziehbarkeit und Robustheit in LLM-gestützten Validierungs- und Korrektursystemen im Produktionsumfeld
Studierende/r:	Ahmad Alsayad
Matrikelnummer:	52318601
Betreuer/in ¹ :	Dipl.-Ing. Michael Macher, BSc, MSc
Version der Einreichung	Version 2

1 Themenbeschreibung

1.1 Einleitung und Problemstellung

Die industrielle Fertigung befindet sich im Zuge der digitalen Transformation in einem tiefgreifenden Wandel. Unter dem Begriff Industry 4.0 werden Produktionsprozesse zunehmend durch vernetzte, datengetriebene Systeme gesteuert, in denen KI-Technologien nicht mehr nur eine analytische Rolle einnehmen, sondern aktiv in operative Entscheidungsprozesse eingebunden sind. Die automatisierte Verarbeitung strukturierter Produktionsdaten, die Unterstützung bei der Fehlererkennung in komplexen Planungssystemen sowie die intelligente Assistenz bei wiederkehrenden, regelbasierten Aufgaben gewinnen dabei zunehmend an Bedeutung – nicht nur als Forschungsgegenstand, sondern als gelebte Praxis in industriellen Projekten.

Im Rahmen eines Praxisprojekts bei der CANCOM Austria AG konnte ich unmittelbar erleben, wie komplex und fehleranfällig die Kombination von KI und Produktionsplanung in der Realität sein kann. Das Produktionsplanungssystem des Kunden verwaltet Fertigungsaufträge, Ressourcenbelegungen und Zeitplanungen, die in Form strukturierter JSON-Snapshot-Dateien abgebildet werden. Diese Snapshots spiegeln den aktuellen Planungszustand wider und bilden die Grundlage für operative Entscheidungen auf dem Shopfloor. In der Praxis enthalten sie jedoch regelmäßige Fehler, die durch wiederholtes

¹ Der Betreuer muss dieses Exposé abgenommen haben bevor es eingereicht werden kann.

manuelles Pflegen an den verschiedensten Stellen entlang der Produktionskette entstehen – angefangen bei der Erfassung von Lagerzuständen, über die Eintragung von Mitarbeiterqualifikationen, bis hin zur Auslieferung von Arbeitsaufträgen an Endkunden. Die manuelle Identifikation und Korrektur dieser Fehler war zeitaufwändig, repetitiv und trotz des Fachpersonals fehleranfällig – ein klarer Ansatzpunkt für KI-gestützte Automatisierung.

Der Einsatz von Large Language Models (LLMs) zur Automatisierung dieses Prozesses erscheint vielversprechend. LLMs können strukturierte Daten analysieren, Muster erkennen und Korrekturen begründen. Erste praktische Erfahrungen zeigen, dass solche Systeme manuelle Bearbeitungszeiten erheblich reduzieren können. Gleichzeitig bringt der LLM-Einsatz in produktionskritischen Umgebungen ein grundlegendes Risiko mit sich, das ich im Projektverlauf selbst beobachten konnte. LLMs produzieren unter bestimmten Bedingungen faktisch inkorrekte, aber syntaktisch plausible Ausgaben – in der Forschungsliteratur als Halluzinationen bezeichnet. (Ji et al., 2023) dokumentieren in ihrer systematischen Übersicht, dass dieses Phänomen kein Randproblem ist, sondern ein strukturelles Merkmal aktueller LLM-Architekturen, das insbesondere bei steigender Aufgabenkomplexität und zunehmender Kontextlänge an Bedeutung gewinnt. Ergänzend zeigen (Wen et al., 2024), dass die Kombination von Retrieval-Mechanismen mit graph-basiertem Reasoning einen vielversprechenden Ansatz darstellt, um sowohl Halluzinationen zu reduzieren als auch die Transparenz von Modellentscheidungen zu verbessern.

Im industriellen Produktionskontext ist dieses Risiko besonders schwerwiegend. Eine syntaktisch valide, aber inhaltlich falsche JSON-Korrektur kann unbemerkt in das ERP-System übernommen werden und dort fehlerhafte Fertigungsaufträge, inkorrekte Ressourcenzuweisungen oder fehlerhafte Terminierungen auslösen – mit direkten operativen Konsequenzen. Hinzu kommen zwei weitere Probleme, die ich in der praktischen Systemarbeit beobachtet habe. Zum einen die mangelnde Nachvollziehbarkeit von Korrekturentscheidungen, die eine effektive Qualitätssicherung erschwert. Zum anderen die mangelnde Robustheit des Systems gegenüber variierenden oder strukturell abweichenden Eingaben, die in heterogenen Produktionsumgebungen alltäglich sind. Diese Unklarheit bei bestimmten Entscheidungen führt dazu, dass die Einhaltung eines Mindestmaßes an Qualität schwer garantierbar erscheint.

Diese drei Problemdimensionen – Halluzinationsrate, Nachvollziehbarkeit und Robustheit – bilden den wissenschaftlichen Ausgangspunkt der vorliegenden Arbeit.

1.2 Projektkontext und bestehendes System

Das empirische Fundament dieser Arbeit ist ein reales Entwicklungsprojekt bei der CANCOM Austria AG, Digital Makers, das ich im Rahmen meines dualen Studiums konzipiert und implementiert habe. Ich war maßgeblich an den technischen Entscheidungen, der Architekturkonzeption und der vollständigen Umsetzung beteiligt, was mir einen unmittelbaren Zugang zu den Systemgrenzen, den praktischen Herausforderungen und den beobachtbaren Verhaltensmustern des Systems gibt. Diese Erfahrung bildet gleichzeitig den Ausgangspunkt für die wissenschaftliche Untersuchung, die in dieser Arbeit durchgeführt wird.

Das Zielsystem: Smart Planning

Smart Planning ist ein Produktionsplanungssystem, das als Integrationslayer zwischen dem ERP-System des Kunden und den operativen Produktionsprozessen fungiert und ebenfalls von der Abteilung Digital Makers entwickelt wurde. Es verwaltet Fertigungsaufträge, Ressourcenpläne, Maschinenbelegungen und Zeitplanungen und stellt diese Daten in Form von JSON-Snapshot-Dateien bereit. Diese Snapshots bilden den aktuellen Planungszustand vollständig ab: Ein einzelner Snapshot kann dabei zwischen 70.000 und über 200.000 Einträge umfassen und enthält Informationen über Produkte, Maschinen-IDs, Produkttypen, Mitarbeiterverfügbarkeiten und -qualifikationen sowie sämtliche produktionsrelevanten Prozessparameter. Auf Basis dieser Snapshot-Daten generiert Smart Planning anschließend den konkreten Produktionsplan – also die operative Steuerung von Fertigungsaufträgen, Ressourcenzuweisungen und Zeitfenstern auf dem Shopfloor. Die Qualität des Snapshots bestimmt damit direkt die Qualität der Produktionsplanung: Fehlerhafte oder inkonsistente Eingangsdaten führen zu fehlerhaften Planungsergebnissen.

Das Smart Planning System verfügt über eine eigene Validierungs-Engine, die Fehler im Snapshot erkennt und lokalisiert. Diese Engine ist jedoch nicht in der Lage, erkannte Fehler automatisch zu korrigieren – eine Aufgabe, die bislang vollständig manuell durch spezialisiertes Fachpersonal ausgeführt wurde und regelmäßig zwei bis über sechs Stunden Arbeitsaufwand pro Snapshot erforderte. Das im Rahmen dieses Projekts entwickelte KI-System setzt genau an dieser Stelle an. Es empfängt fehlerhafte Snapshots aus Smart Planning, analysiert und korrigiert diese mithilfe eines LLM-gestützten Agentensystems und spielt die korrigierten Snapshots anschließend automatisiert zurück ins System. Smart Planning fungiert dabei sowohl als Datenquelle als auch als Ziel der Korrekturausgabe – das KI-System ist damit direkt in jenen Prozessschritt eingebettet, der dem eigentlichen Planungslauf vorausgeht.

Das entwickelte Vier-Agenten-System

Das System realisiert eine automatisierte Korrekturpipeline auf Basis eines Vier-Agenten-Ansatzes mit Azure OpenAI GPT-4.1. Die Architektur gliedert sich in einen Orchestrator-Agenten und drei spezialisierte Unteragenten. Die gesamte Infrastruktur ist in der Azure-Cloud deployed und wird über Terraform als Infrastructure-as-Code verwaltet.

Der Orchestrator-Agent ist die primäre Schnittstelle zum Endnutzer. Er empfängt Anfragen, analysiert die Nutzerintention, plant die notwendigen Verarbeitungsschritte und koordiniert die Aufgabenverteilung an die Unteragenten. Er entscheidet dynamisch, welcher Agent für welche Teilaufgabe zuständig ist, und konsolidiert die Rückmeldungen zu einer logisch stimmigen Antwort. Das zentrale Designprinzip dabei ist die kontextsensitive Entscheidungsfindung: Der Orchestrator arbeitet nicht nach fest programmierten if-then-Regeln, sondern trifft seine Routing-Entscheidungen auf Basis von LLM-Calls mit einem spezialisierten Systemprompt. Die Qualität dieser Koordination – insbesondere die Fähigkeit, mehrstufige Korrekturprozesse zu planen und bei Fehlschlägen alternative Strategien zu wählen – erwies sich im Projektverlauf als eine der zentralen praktischen Herausforderungen.

Der RAG-Agent greift über Azure AI Search auf eine strukturierte Wissensbasis aus Produktionsdokumenten und Validierungsregelwerken zu. Die Korrekturregeln sind dabei bewusst in natürlicher Sprache verfasst – nicht als Code oder hard-kodierte if-then-Logik –

sodass Fachexperten ohne Programmierkenntnisse die Regeln erstellen und pflegen können. Die Qualität der Retrieval-Ergebnisse hat sich als entscheidender Faktor für die inhaltliche Korrektheit der Systemausgaben erwiesen.

Der Smart-Planning-Agent ist die operative Kernkomponente des Systems. Er verfügt über ein dediziertes Toolset aus zehn spezialisierten Werkzeugen und vier vordefinierten Pipelines für wiederkehrende Workflows. Diese Tools ermöglichen es ihm, Snapshots direkt aus der Software Smart Planning abzuholen, sie mit der systemseitigen Validierungs-Engine zu prüfen, Fehler mithilfe von LLM-Aufrufen zu analysieren, strukturierte Korrekturvorschläge zu generieren, diese auf die Snapshot-Daten anzuwenden und die korrigierten Snapshots schließlich automatisiert zurückzuladen. Er ist damit der einzige Agent, der direkt mit den JSON-Strukturen arbeitet und die eigentliche Korrekturlogik ausführt. Da Snapshots mit 70.000 bis über 200.000 Einträgen zu groß sind, um vollständig an ein LLM übergeben zu werden, arbeitet der Agent mit gezielten Kontextextraktionen, die typischerweise 1.000 bis 4.000 relevante Zeilen umfassen.

Der Chat-Agent verfügt über initiales Trainingswissen zu allgemeinen Themen, wie Produktionsregeln und typischen Fehlermustern. Er übernimmt Aufgaben, die weder spezifisches Retrieval noch eine direkte Systemintegration erfordern.

Die Snapshot-Korrektur-Pipeline

Der operative Ablauf des Systems folgt einem klar definierten, aber inhärent iterativen Muster. Der Smart-Planning-Agent holt den aktuellen Snapshot aus dem Zielsystem ab, übergibt ihn an die Validierungs-Engine, überführt das Ergebnis in eine LLM-gestützte Fehleranalyse, generiert einen strukturierten Korrekturvorschlag, wendet diesen an und validiert erneut. Da Korrekturen häufig neue Inkonsistenzen aufdecken – eine behobene fehlende ID kann etwa zu einem neu entstehenden Duplikat führen – wiederholt sich dieser Zyklus iterativ, bis der Snapshot den Status „valide“ erreicht oder eine definierte Maximalanzahl an Iterationen überschritten wird. Jede Änderung wird dabei in einer Metadaten-Datei protokolliert und am Ende in einem revisionssicheren Audit-Report zusammengefasst. In der Praxis erforderten typische Snapshots mit drei bis fünf Fehlern eine Gesamtbearbeitungszeit von ein bis drei Minuten – gegenüber dem bisherigen manuellen Aufwand von zwei bis sechs Stunden entspricht das einer Zeitersparnis von 95 bis 99 Prozent. Der Orchestrator koordiniert diesen Prozess auf Kommunikationsebene und stellt sicher, dass Nutzeranfragen korrekt an den zuständigen Agenten delegiert werden.

Der monolithische Systemprompt-Ansatz

Das bestehende System basiert auf monolithischen Systemprompts. Die gesamte Verarbeitungslogik jedes Agenten – Rollendefinition, Validierungsregeln, Korrekturanweisungen, Ausgabeformatierung und Beispielfälle – ist in einem einzigen, umfangreichen Prompt-Dokument gebündelt. Der aktuelle Systemprompt des Smart-Planning-Agenten umfasst 425 Zeilen und 20.284 Zeichen. Dieser Ansatz war in der initialen Entwicklungsphase pragmatisch und schnell umsetzbar, da er keine vorgelagerte Architekturentscheidung erforderte und eine schnelle Iteration über Prompt-Inhalte ermöglichte. Mit wachsender Komplexität der verarbeiteten Snapshots und steigenden Qualitätsanforderungen treten jedoch strukturelle Schwächen auf, die ich im Projektverlauf direkt beobachten konnte und die im folgenden Abschnitt systematisch beschrieben werden.

1.3 Problemstellung

Die im Projektverlauf beobachteten Schwächen des monolithischen Ansatzes lassen sich in drei klar abgrenzbare Problemdimensionen gliedern. Diese Probleme sind nicht isoliert zu betrachten – sie bedingen und verstärken sich gegenseitig – und bilden gemeinsam den wissenschaftlichen Kern, den diese Arbeit adressiert.

Problem 1: Halluzinationen bei komplexen Validierungsaufgaben

In der praktischen Arbeit mit dem System zeigte sich ein konsistentes Muster. Bei einfachen, gut strukturierten Snapshots mit wenigen, klar lokalisierbaren Fehlern liefert das System zuverlässige Korrekturen. Bei komplexen Snapshots hingegen – mit vielen verschachtelten Abhängigkeiten, mehreren gleichzeitigen Fehlern oder unvollständigen Referenzen – steigt die Rate fachlich inkorrektur Ausgaben an. Korrekturen, die syntaktisch valide erscheinen, aber inhaltlich falsche Werte oder nicht belegbare Zustandsänderungen enthalten, wurden dabei wiederholt beobachtet. Besonders ausgeprägt war dieses Verhalten in Situationen, in denen das LLM Korrekturvorschläge mit Referenzen oder Annahmen generierte, die nicht den dokumentierten Unternehmensregeln entsprachen – auch dann, wenn diese Regeln explizit im Systemprompt oder im RAG-Kontext hinterlegt waren. Die Messbarkeit dieser Fehlerrate ist dabei selbst eine methodische Herausforderung, die diese Arbeit adressiert: Da keine automatisierte Ground-Truth-Validierung existiert, erfordert die Beurteilung, ob eine Korrektur fachlich korrekt ist, domänenspezifisches Expertenwissen – was zugleich die Notwendigkeit einer expertenbasierten Evaluierungsmethode begründet.

Die Ursache liegt in einer bekannten Eigenschaft monolithischer Prompts. Je mehr Validierungsregeln, Kontextinformationen und Korrekturanweisungen gleichzeitig in einem einzigen Kontext verarbeitet werden müssen, desto wahrscheinlicher ist es, dass das Modell relevante Constraints vernachlässigt oder widersprüchliche Regeln fehlerhaft auflöst. (Ji et al., 2023) beschreiben dieses Muster als charakteristisch für LLM-Systeme, bei denen Aufgabenkomplexität und Kontextlänge eine kritische Schwelle überschreiten. Im vorliegenden Anwendungsfall ist dieses Problem besonders kritisch, weil eine fachlich falsche, aber syntaktisch valide JSON-Korrektur von nachgelagerten Validierungsstufen möglicherweise nicht erkannt und unbemerkt in das ERP-System übernommen werden kann.

Problem 2: Fehlende Nachvollziehbarkeit von Korrekturentscheidungen

Ein Problem, das mir im Projektalltag besonders deutlich wurde, ist die fehlende Transparenz der Entscheidungslogik bei fehlerhaften Ausgaben. Wenn das System korrekte Korrekturen liefert, begründet es seine Entscheidungen – da dies als explizite Anforderung im Systemprompt hinterlegt ist. Das eigentliche Problem tritt auf, wenn Korrekturen inhaltlich falsch sind und die mitgelieferte Begründung nicht nachvollziehbar ist oder auf Regelreferenzen verweist, die sich beim näheren Hinsehen als nicht existent oder falsch interpretiert erweisen. In solchen Fällen ist weder als Entwickler noch als Fachperson rekonstruierbar, an welcher Stelle im Systemprompt oder im Entscheidungsprozess der Fehler entstanden ist.

Der Ansatz zur Verbesserung wäre im Prinzip klar: Man müsste die fehlerhafte Ausgabe analysieren, den gedanklichen Pfad rekonstruieren, der zu dieser Entscheidung geführt hat,

und die entsprechende Stelle im Systemprompt gezielt korrigieren. In der Praxis ist das mit einem monolithischen Prompt jedoch kaum möglich. Da alle Verarbeitungsschritte in einem einzigen, 425 Zeilen umfassenden Dokument verschmelzen, gibt es keine expliziten Zwischenzustände, keine dokumentierten Entscheidungspunkte und keinen rekonstruierbaren Pfad. Was bleibt, ist ein Korrekturversuch auf Basis von Vermutungen: Man identifiziert eine Stelle im Prompt, die plausibel für das beobachtete Fehlverhalten verantwortlich sein könnte, und passt sie an – ohne zu wissen, ob man damit tatsächlich die Ursache adressiert. Das eigentliche Problem mag dadurch behoben werden oder auch nicht. Was in jedem Fall hinzukommt, sind potenzielle Nebeneffekte an anderen Stellen des Prompts, die zuvor stabil funktionierten: Eine Anpassung, die einen Fehler beseitigt, kann gleichzeitig Verhaltensweisen destabilisieren, die bereits als robust galten – und diese Destabilisierung bleibt im schlimmsten Fall unbemerkt, weil auch hier keine expliziten Zwischenzustände existieren, die auf Veränderungen hinweisen würden. Mit zunehmender Reife und Komplexität des Prompts verschärft sich dieses Problem: Je sorgfältiger das bestehende Verhalten ausdefiniert wurde, desto mehr potenzielle Angriffspunkte entstehen bei jeder weiteren Änderung. Robustheit ist in diesem Ansatz kein erreichter Zustand, sondern muss bei jedem Eingriff neu erprobt und verifiziert werden. (Besta et al., 2024) zeigen, dass die explizite Modellierung von Reasoning-Schritten als Graph genau hier ansetzt – indem Zwischenergebnisse sichtbar, validierbar und gezielt korrigierbar werden, ohne dass eine Anpassung an einer Stelle unkontrolliert in andere Verarbeitungsschritte ausstrahlt.

Problem 3: Mangelnde Robustheit bei variierenden Eingaben

ERP-Snapshots in produktionsnahen Umgebungen sind selten konsistent strukturiert. Im Projektverlauf begegneten mir regelmäßig Snapshots mit fehlenden Pflichtfeldern, unbekannten Feldwerten, inkonsistenten Referenzen oder strukturellen Abweichungen – Grenzfälle, die das System vor besondere Herausforderungen stellen. Das beobachtete Verhalten in solchen Situationen war instabil: Wiederholte Anfragen mit identischen Eingaben lieferten mitunter unterschiedliche Korrekturen, und stark abweichende Snapshots wurden ohne adäquate Fehlererkennung verarbeitet. Erschwerend kommt hinzu, dass Korrekturen häufig Kaskadeneffekte auslösen – eine behobene Inkonsistenz deckt eine weitere auf, die im vorherigen Durchlauf nicht sichtbar war – was die iterative Korrekturschleife verlängert und die Vorhersehbarkeit des Gesamtprozesses weiter reduziert.

Es ist dabei wichtig, zwischen zwei Arten von Nichtdeterminiertheit zu unterscheiden. Zum einen ist eine gewisse Variabilität in den Formulierungen von LLM-Ausgaben systemimmanent und unvermeidbar – sie liegt in der stochastischen Natur der Modelle begründet. Zum anderen gibt es eine inhaltliche Instabilität, bei der dieselben Symptome zu unterschiedlichen fachlichen Korrekturen führen – obwohl die zugrundeliegenden Fehler bekannt, dokumentiert und im Systemprompt beschrieben sind. Genau diese zweite Form der Instabilität ist in einem produktionskritischen System nicht akzeptabel. Die These dieser Arbeit ist, dass graph-basierte Architekturen durch die explizite Modellierung von Verarbeitungsschritten und Zustandsübergängen zu konsistenteren Korrekturen führen können – nicht weil sie die stochastische Natur von LLMs eliminieren, sondern weil sie den Entscheidungsraum für jeden Teilschritt gezielter eingrenzen.

Die wissenschaftliche Lücke

Die beschriebenen Probleme sind in der Forschungsliteratur grundsätzlich bekannt. Halluzinationsprobleme in LLMs sind gut dokumentiert (Ji et al., 2023), und die potenziellen Vorteile graph-basierter Reasoning-Strukturen werden in aktuellen Arbeiten überzeugend dargestellt (Besta et al., 2024; Wen et al., 2024). Was jedoch fehlt, ist der Nachweis, ob diese Vorteile auch in einem realen, industriellen Anwendungsfall mit strukturierten, domänenspezifischen Ausgaben – konkret JSON-Korrekturen in einem produktionskritischen System – tatsächlich realisiert werden können.

Bestehende Arbeiten zu Graph-of-Thoughts evaluieren diese Architekturen überwiegend an abstrakten Reasoning-Benchmarks oder generischen Sprachverständnisaufgaben. Die Übertragbarkeit auf produktionskritische Systeme mit komplexen strukturierten Ausgaben und strengen Anforderungen an Revisionssicherheit und Nachvollziehbarkeit ist bislang nicht empirisch untersucht worden. Ebenso fehlt es an methodischen Beiträgen, die konkrete Evaluierungsansätze für solche Ausgaben jenseits klassischer Text-Metriken entwickeln. Diese doppelte Lücke – empirisch und methodisch – zu schließen ist das zentrale wissenschaftliche Anliegen der vorliegenden Arbeit.

1.4 Zielsetzung der Arbeit

Aus der beschriebenen Problemstellung und der identifizierten wissenschaftlichen Lücke leiten sich drei klar unterscheidbare Zielsetzungen ab, die gemeinsam den Beitrag dieser Arbeit definieren.

Wissenschaftliches Ziel: empirischer Architekturvergleich

Das primäre Ziel ist ein methodisch kontrollierter, empirisch fundierter Vergleich zwischen dem bestehenden monolithischen System und einer neu konzipierten Variante mit graph-basierter Architektur. Dieser Vergleich soll auf Basis operationalisierbarer Metriken quantifizierbare Aussagen darüber liefern, ob und in welchem Ausmaß der graph-basierte Ansatz in den drei zentralen Dimensionen – Halluzinationsrate, Nachvollziehbarkeit und Robustheit – Verbesserungen erzielt. Dabei ist ausdrücklich nicht das Ziel, eine allgemeingültige Aussage über LLM-Architekturen zu treffen, sondern einen empirisch fundierten Beitrag für den spezifischen Kontext produktionskritischer Validierungssysteme mit strukturierten Ausgaben zu leisten.

Methodisches Ziel: Evaluierungsrahmen für strukturierte LLM-Ausgaben

Ein eigenständiges Ziel ist die Entwicklung eines geeigneten Evaluierungsrahmens für komplexe, strukturierte LLM-Ausgaben. Etablierte Frameworks wie RAGAS sind primär für textbasierte Antworten konzipiert und lassen sich nicht ohne Weiteres auf JSON-Korrekturen anwenden. Da ich im Projektverlauf selbst erfahren habe, wie schwierig es ist, die Qualität solcher Ausgaben objektiv und nachvollziehbar zu beurteilen – insbesondere in Situationen, in denen eine Korrektur syntaktisch korrekt, inhaltlich aber falsch ist – sehe ich hierin einen eigenständigen Beitrag dieser Arbeit. Sie entwickelt und erprobt eine Kombination aus automatisierten Metriken, domänenexpertenbasierter Bewertung und struktureller

Validierung, die als übertragbarer methodischer Beitrag für ähnliche Evaluierungsaufgaben nutzbar sein soll.

Praktisches Ziel: produktionsnahe Systemarchitektur

Das praktische Ziel ist die prototypische Implementierung einer graph-basierten Variante des bestehenden Systems sowie die Ableitung konkreter Designrichtlinien für den Einsatz von Prompt-Architekturen in industriellen KI-Systemen. Das Ergebnis soll unmittelbar als Grundlage für eine Weiterentwicklung des bestehenden Systems nutzbar sein.

Gesamtbeitrag

Die Arbeit leistet einen dreifachen Beitrag: einen empirischen durch den kontrollierten Vergleich zweier Architekturansätze an einem realen Industrieanwendungsfall, einen methodischen durch die Entwicklung eines Evaluierungsrahmens für strukturierte LLM-Ausgaben in produktionsnahen Systemen, und einen praktischen in Form einer weiterentwickelten Systemarchitektur mit daraus abgeleiteten Designrichtlinien für den Einsatz von Prompt-Architekturen in industriellen KI-Systemen.

2 Forschungsfrage / Forschungsziel / Thesen

2.1 Hauptforschungsfrage

Die zentrale Forschungsfrage dieser Arbeit lautet:

Inwiefern unterscheidet sich eine graph-basierte Systemarchitektur von einer monolithischen Systemprompt-Struktur hinsichtlich Halluzinationsrate, Nachvollziehbarkeit und Robustheit bei der automatisierten Validierung und Korrektur strukturierter JSON-Daten in einem produktionskritischen Umfeld?

Diese Frage ist bewusst komparativ angelegt. Sie zielt nicht auf eine abstrakte Bewertung von Prompt-Architekturen im Allgemeinen, sondern auf einen fundierten Vergleich zweier konkreter Implementierungen an einem realen industriellen Anwendungsfall. Die drei Dimensionen – Halluzinationsrate, Nachvollziehbarkeit und Robustheit – entsprechen dabei direkt den in Abschnitt 1.3 beschriebenen Problemdimensionen und bilden gleichzeitig die Grundlage für das Evaluierungsdesign.

Aus der Hauptforschungsfrage ergeben sich drei Unterfragen, die jeweils einen spezifischen Aspekt des Vergleichs operationalisieren:

2.1.1 Unterfragen (zur Ergänzung/Vertiefung)

- Welchen Einfluss hat die Modularisierung von Systemprompts in graph-basierte Workflow-Strukturen auf die Reduktion von Halluzinationen in automatisierten Fehlerkorrekturprozessen?

Im Mittelpunkt steht dabei nicht nur die Frage, ob graph-basierte Architekturen weniger halluzinieren, sondern auch, wie dieser Effekt – sofern er existiert – überhaupt messbar gemacht werden kann. Da klassische Evaluierungsframeworks wie RAGAS primär für offene Textantworten konzipiert sind und sich nicht ohne Weiteres auf strukturierte JSON-Korrekturen mit dokumentierten Regelwerken übertragen lassen, ist die Entwicklung eines geeigneten Messansatzes selbst Teil der Untersuchung. UF1 verbindet damit die inhaltliche Dimension des Architekturvergleichs mit dem methodischen Beitrag dieser Arbeit.

- Inwiefern beeinflusst die gewählte Prompt-Architektur die Konsistenz und Stabilität von Systementscheidungen bei variierenden Eingabedaten im Vergleich zu einem monolithischen Systemprompt?

In der praktischen Systemarbeit zeigte sich, dass identische oder ähnliche Eingaben unter dem monolithischen Ansatz mitunter zu unterschiedlichen fachlichen Korrekturen führten – ein Verhalten, das in produktionskritischen Umgebungen nicht tolerierbar ist. UF2 untersucht, ob die explizite Zerlegung des Verarbeitungsprozesses in diskrete, graph-modellierte Schritte zu konsistenteren Entscheidungen führt, insbesondere bei Grenzfällen wie fehlenden Pflichtfeldern, inkonsistenten Referenzen oder strukturell abweichenden Snapshots. Dabei wird auch die Frage berücksichtigt, inwieweit beobachtete Inkonsistenzen auf die stochastische Natur des Modells zurückzuführen sind – und inwieweit sie architekturbedingt und damit durch einen strukturierten Ansatz adressierbar sind.

- Wie unterscheiden sich beide Ansätze hinsichtlich Debugging, Wartbarkeit und Fehlertoleranz bei iterativen Korrekturschleifen in produktionsnahen Fertigungsumgebungen?

Im Projektverlauf zeigte sich, dass Anpassungen am monolithischen Systemprompt einem Blindflug gleichen: Da keine expliziten Zwischenzustände existieren, lässt sich nicht zuverlässig bestimmen, an welcher Stelle eine Korrektur ansetzen muss – und jede Änderung birgt das Risiko, andernorts stabil funktionierendes Verhalten zu destabilisieren. UF3 untersucht daher, ob graph-basierte Architekturen durch die explizite Sichtbarmachung von Verarbeitungsschritten und Entscheidungspunkten eine gezieltere Fehleranalyse und wartungsfreundlichere Weiterentwicklung ermöglichen. Besonders relevant ist dieser Aspekt im Kontext iterativer Korrekturschleifen, bei denen eine Korrektur häufig neue Inkonsistenzen aufdeckt und der Prozess mehrfach durchlaufen wird.

3 Abgrenzung der Arbeit

Die vorliegende Arbeit fokussiert sich auf einen klar definierten Untersuchungsgegenstand und grenzt sich von angrenzenden Fragestellungen ab, um wissenschaftliche Präzision und methodische Handhabbarkeit zu gewährleisten.

Der Vergleich beschränkt sich auf zwei konkrete Systemarchitekturen – den bestehenden monolithischen Systemprompt und eine neu konzipierte graph-basierte Systemarchitektur – im Kontext eines spezifischen Anwendungsfalls: die automatisierte Validierung und Korrektur von JSON-Snapshot-Dateien im Produktionsplanungssystem Smart Planning. Ausgangspunkt ist dabei das bereits bestehende Vier-Agenten-System aus dem Praxisprojekt, das den praktischen Untersuchungsgegenstand bildet. Die graph-basierte Systemarchitektur wird als Vergleichsvariante konzipiert und prototypisch untersucht. Ziel ist daher nicht die vollständige Produktivsetzung einer neuen Lösung, sondern die wissenschaftlich nachvollziehbare Bewertung, ob und in welchen Bereichen eine graph-basierte Struktur gegenüber einer monolithischen Systemprompt-Struktur Vorteile bietet.

Unter einer graph-basierten Systemarchitektur wird in dieser Arbeit nicht nur die Struktur einzelner Prompts verstanden, sondern die strukturierte Modellierung von Daten, Systemprompts, Teilprompts, Regeln, Beispieldaten, Kontextinformationen, Tool-Ausgaben, Zwischenergebnissen und fachlichen Entscheidungsgrundlagen als gerichteter Verarbeitungsgraph. Da der Begriff „Prompt-Architektur“ in der Forschungsliteratur nicht einheitlich verwendet wird – (Besta et al., 2024; Wen et al., 2024) sprechen von graph-basiertem Reasoning und graph-basierter Wissensmodellierung, nicht von Prompt-Architekturen im engeren Sinne, wird er in dieser Arbeit bewusst nicht als etablierter wissenschaftlicher Fachbegriff vorausgesetzt, sondern durch den präziseren Begriff der graph-basierten Systemarchitektur ersetzt und auf den beschriebenen Anwendungskontext eingegrenzt.

Kein Bestandteil dieser Arbeit ist die Grundlagenforschung an Large Language Models selbst. Es wird nicht untersucht, welches Sprachmodell grundsätzlich am leistungsfähigsten ist oder welches Foundation Model für industrielle Anwendungen am besten geeignet wäre. Das zugrundeliegende Modell, Azure OpenAI GPT-4.1, bleibt in beiden Vergleichsvarianten identisch, damit beobachtbare Unterschiede primär auf die Systemstruktur und nicht auf Modellunterschiede zurückgeführt werden können.

Die Arbeit grenzt sich außerdem von einer reinen Prompt-Optimierung ab. Es geht nicht primär darum, einzelne Formulierungen innerhalb eines Prompts zu verbessern. Vielmehr wird untersucht, ob die Zerlegung eines komplexen Validierungs- und Korrekturprozesses in mehrere definierte Verarbeitungsschritte zu besseren und nachvollziehbaren Ergebnissen führt. Der Schwerpunkt liegt damit auf der Systemstruktur und dem Entscheidungsfluss, nicht auf rein sprachlicher Prompt-Formulierung.

Der entwickelte Evaluierungsrahmen erhebt keinen Anspruch auf universelle Übertragbarkeit und grenzt sich von klassischen Validierungsansätzen ab. Klassische Validierungsmechanismen können prüfen, ob ein JSON syntaktisch korrekt ist, ob ein Schema eingehalten wird oder ob bestimmte Pflichtfelder vorhanden sind.

Solche Prüfungen sind notwendig, reichen für die Forschungsfrage jedoch nicht aus. Sie können nicht beurteilen, ob eine vorgeschlagene Korrektur fachlich sinnvoll ist, ob sie im Produktionskontext angemessen ist oder ob sie neue Folgefehler auslösen kann. Deshalb wird technische Validierung in dieser Arbeit mit fachlicher Expertenbewertung und kontrollierten Testfällen kombiniert.

Auch automatisierte Evaluierungsframeworks wie RAGAS werden nicht als alleinige Bewertungsgrundlage verstanden. RAGAS ist primär für klassische Retrieval-Augmented-Generation-Szenarien geeignet, bei denen eine natürlichsprachliche Antwort anhand eines abgerufenen Kontextes bewertet wird. Im vorliegenden System entstehen jedoch nicht nur klassische Textantworten, sondern auch strukturierte JSON-Korrekturvorschläge, Validierungsergebnisse, Tool-Ausgaben und operative Systementscheidungen, die durch klassische RAG-Metriken nur teilweise erfasst werden können. RAGAS kann daher ergänzend eingesetzt werden, ersetzt jedoch keine technische Validierung und keine fachliche Expertenbewertung.

Die Untersuchung beschränkt sich zudem auf ausgewählte, repräsentative Fehler- und Testfälle aus dem Produktionsplanungs- und Smart-Planning-Kontext. Dazu können beispielsweise leere Pflichtfelder, ungültige Referenzen, inkonsistente Datenbeziehungen, fehlerhafte Korrekturwerte oder unvollständige Snapshot-Daten gehören. Es ist nicht Ziel der Arbeit, alle denkbaren Fehlerfälle industrieller Produktionsplanung vollständig abzudecken. Stattdessen sollen anhand ausgewählter Testfälle belastbare Aussagen über den Vergleich der beiden Systemansätze ermöglicht werden.

Zusammenfassend untersucht die Arbeit nicht allgemein, ob LLMs in der Produktion sinnvoll eingesetzt werden können, und auch nicht ausschließlich, ob bessere Prompts bessere Antworten erzeugen. Der konkrete Fokus liegt auf der Frage, ob eine graph-basierte Systemarchitektur gegenüber einer monolithischen Systemprompt-Struktur in einem LLM-gestützten Validierungs- und Korrektursystem messbare Vorteile hinsichtlich Halluzinationsrate, Nachvollziehbarkeit und Robustheit bietet.

4 Wissenschaftliche Methodik und Forschungsdesign

Für diese Bachelorarbeit wird ein empirisch-vergleichendes Forschungsdesign gewählt. Im Mittelpunkt steht der kontrollierte Vergleich zweier Systemansätze innerhalb eines realen, produktionsnahen LLM-Systems: einer monolithischen Systemprompt-Struktur und einer graph-basierten Systemarchitektur. Ziel ist es, nicht nur theoretisch zu argumentieren, dass eine graph-basierte Struktur Vorteile haben könnte, sondern diese Annahme anhand konkreter Testfälle, messbarer Kriterien und fachlicher Bewertungen zu überprüfen. Die Arbeit ist als praxisnahe Fallstudie mit experimentellen Vergleichselementen angelegt, wobei das bestehende Vier-Agenten-System aus dem Smart-Planning-Kontext den praktischen Untersuchungsgegenstand bildet.

4.1 Untersuchungsaufbau und Vergleichsdesign

Methodisch wird zunächst der bestehende monolithische Ansatz systematisch analysiert – hinsichtlich der darin gebündelten Aufgaben wie Rollenbeschreibungen, fachliche Regeln, Validierungslogiken, Entscheidungsanweisungen und Ausgabeformatierung. Diese Analyse bildet die empirisch fundierte Baseline. Darauf aufbauend wird eine graph-basierte Systemarchitektur als Vergleichsvariante konzipiert, bei der die bisher gebündelte Entscheidungs- und Verarbeitungslogik in klar definierte Schritte zerlegt wird – etwa Eingabeanalyse, Fehlerklassifikation, Kontextsuche, Regelzuordnung, Korrekturgenerierung und Ergebnisbewertung. Diese Schritte werden als Knoten eines Graphen modelliert, Abhängigkeiten und Kontrollflüsse als Kanten. Der Ansatz orientiert sich an den theoretischen Grundlagen von (Besta et al., 2024), die mit Graph of Thoughts zeigen, dass Zwischenergebnisse von LLM-Prozessen explizit als Graphstruktur modelliert werden können, sowie von (Wen et al., 2024), die graph-basierte Wissensstrukturen zur Verbesserung von Transparenz und Nachvollziehbarkeit einsetzen. (Ji et al., 2023) liefern die Grundlage zur Einordnung von Halluzinationen und begründen, warum deren Bewertung nicht allein durch klassische automatische Metriken abgedeckt werden kann. Für den Vergleich wird ein Testfallkatalog mit repräsentativen Fehlerszenarien aus dem Smart-Planning-Kontext erstellt – etwa leere Pflichtfelder, ungültige Referenzen, inkonsistente Datenbeziehungen oder unvollständige Snapshot-Daten. Beide Varianten werden mit identischen Eingaben konfrontiert, bei konstantem Modell, konstanten Modellparametern und randomisierter Ausführungsreihenfolge, damit beobachtbare Unterschiede primär auf die Systemstruktur zurückgeführt werden können.

4.2 Bewertungsmethoden und Erkenntnisziel

Die Bewertung der Systemausgaben kombiniert quantitative und qualitative Methoden, da die Qualität strukturierter JSON-Korrekturen in einem domänenspezifischen Produktionskontext nicht allein durch eine technische Metrik erfasst werden kann. Technische Prüfungen – JSON-Validierung, Schema-Prüfung, Referenzvalidierung und Re-Validierung des korrigierten Snapshots – liefern quantifizierbare Ergebnisse wie Fehlerquoten, die Anzahl ungültiger Ausgaben und die Konsistenz bei Wiederholungstests.

Ergänzt werden diese durch eine fachliche Expertenbewertung. Mindestens zwei bis vier Personen aus dem Projekt- oder Kundenumfeld bewerten Systemausgaben anhand eines einheitlichen Rasters hinsichtlich fachlicher Korrektheit, Nachvollziehbarkeit, technischer Verwendbarkeit und potenzieller Folgefehler. Die Expertengespräche, Reviews und Austauschrunden werden im Rahmen der Arbeit systematisch protokolliert. Diese Protokolle dienen als qualitative Datenquelle für die spätere Auswertung.

Zusätzlich werden Nutzertests nach SUS und UEQ mit mindestens fünf Teilnehmenden durchgeführt, um insbesondere die wahrgenommene Nachvollziehbarkeit und Verlässlichkeit beider Varianten zu erfassen. RAGAS wird ergänzend für jene Teilaspekte eingesetzt, bei denen textbasierte RAG-Metriken aussagekräftige Hinweise liefern können, etwa bei der Bewertung abgerufener Kontextinformationen oder natürlichsprachlicher Begründungen (Es et al., 2024). Für JSON-Korrekturvorschläge, Tool-Ausgaben und operative Systementscheidungen reichen klassische RAG-Metriken jedoch nicht aus und werden daher durch technische Validierung sowie fachliche Expertenbewertung ergänzt.

Die Auswertung erfolgt vergleichend entlang der drei Qualitätsdimensionen Halluzinationsrate, Nachvollziehbarkeit und Robustheit. Ziel ist eine differenzierte Aussage darüber, unter welchen Bedingungen eine graph-basierte Systemarchitektur für LLM-gestützte Validierungs- und Korrektursysteme sinnvoll ist. Die Methodik ist damit an der praktischen Realität des Projekts ausgerichtet: Aus der eigenen Projekterfahrung heraus zeigt sich, dass die zentrale Herausforderung nicht nur darin besteht, ein funktionierendes KI-System zu bauen, sondern dessen Entscheidungen überprüfbar und belastbar zu machen.

5 Evaluierungsdesign

Das Evaluierungsdesign bildet den methodischen Kern dieser Arbeit, da hier festgelegt wird, wie die beiden Systemansätze nachvollziehbar miteinander verglichen werden. Ziel ist es, nicht nur qualitativ zu beschreiben, dass eine graph-basierte Systemarchitektur plausibler oder transparenter erscheint, sondern auf Basis konkreter Bewertungsdaten entscheiden zu können, ob sie gegenüber der monolithischen Systemprompt-Struktur tatsächlich messbare Vorteile aufweist. Bewertet wird dabei nicht nur, ob am Ende ein valider Snapshot entsteht, sondern auch, wie dieser Zustand erreicht wurde – denn eine Korrektur kann technisch erfolgreich, aber fachlich schlecht begründet oder schwer nachvollziehbar sein. Diese Unterscheidung zwischen Ergebnisqualität und Prozessqualität ist für den vorliegenden Anwendungsfall entscheidend.

5.1 Operationalisierung der Evaluierungsdimensionen

Die drei Qualitätsdimensionen – Halluzinationsrate, Nachvollziehbarkeit und Robustheit – werden für den vorliegenden Anwendungsfall konkret definiert, bevor Messungen durchgeführt werden.

Halluzinationsrate bezeichnet den Anteil der Systemausgaben, die fachlich inkorrekte, nicht belegbare oder regelwidrige Korrekturen enthalten – unabhängig davon, ob diese syntaktisch valide sind. Halluzinationen können dabei unterschiedliche Formen annehmen. Erfundene Regelreferenzen, fachlich falsche Korrekturwerte, nicht belegbare Annahmen über Datenbeziehungen, ungültige JSON-Strukturen oder Korrekturen, die im Zielsystem nicht ausführbar sind. Für jeden Testfall wird daher kategorisiert, welche Art von Fehler vorliegt – fachliche Halluzination, strukturelle Halluzination, Regelhalluzination oder Folgefehlererzeugung. Diese Differenzierung ist wichtig, weil nicht jede Halluzination automatisch durch technische Validierung erkannt werden kann. Eine JSON-Korrektur kann formal gültig und dennoch fachlich falsch sein – genau diese Fälle sind im industriellen Kontext schwerer zu erkennen und potenziell riskanter.

Nachvollziehbarkeit bezeichnet das Ausmaß, in dem der Weg von der Eingabe zur Ausgabe rekonstruiert werden kann. Bewertet wird, ob klar erkennbar ist, welcher Fehler erkannt wurde, welche Regel angewendet wurde, welche Daten zur Entscheidung herangezogen wurden und warum eine bestimmte Korrektur vorgeschlagen wurde. Eine Begründung gilt dabei nur dann als nachvollziehbar, wenn sie den realen Entscheidungsprozess korrekt

abbildet – nicht nur nachträglich plausibel klingt (Potenzielle Verbesserung der Korrekturlogik).

Robustheit bezeichnet die Fähigkeit des Systems, bei strukturell variierenden oder fehlerhaften Eingaben konsistente und fachlich angemessene Ausgaben zu erzeugen. Eine robuste Systemvariante zeichnet sich dabei nicht dadurch aus, dass sie immer eine Korrektur erzwingt – in produktionskritischen Systemen kann es qualitativ hochwertiger sein, eine Unsicherheit transparent auszuweisen, statt eine scheinbar plausible, aber fachlich nicht abgesicherte Korrektur zu erzeugen.

5.2 Testfallkatalog

Der Testfallkatalog enthält repräsentative Fehlerszenarien aus dem Smart-Planning-Kontext, gegliedert nach Komplexität. Standardfälle umfassen klar definierte Einzelfehler mit eindeutiger Korrekturanweisung im Regelwerk – sie bilden die Basis für die Messung der Halluzinationsrate unter kontrollierten Bedingungen. Komplexfälle umfassen Szenarien mit mehreren gleichzeitigen Fehlern, verschachtelten Abhängigkeiten oder widersprüchlichen Feldinhalten, bei denen die Korrektur einer Stelle Auswirkungen auf andere Stellen hat.

5.3 Bewertungsverfahren

Die Bewertung erfolgt auf drei Ebenen. Auf der technischen Ebene werden Ausgaben automatisiert geprüft. JSON-Schema-Validierung, Referenzprüfung, Pflichtfeldprüfung und Re-Validierung des korrigierten Snapshots durch die Smart-Planning-Validierungs-Engine. Quantitative Kennzahlen umfassen Anzahl gültiger und ungültiger Korrekturen, erfolgreiche Re-Validierungen, neu entstandene Folgefehler und benötigte Korrekturiterationen. Ergänzend werden Wiederholungstests mit identischen Eingaben durchgeführt, um inhaltliche Variabilität zu erfassen.

Auf der fachlichen Ebene bewerten mindestens zwei bis vier Expertinnen und Experten aus dem Projekt- oder Kundenumfeld ausgewählte Systemausgaben anhand eines einheitlichen Bewertungsrasters – hinsichtlich fachlicher Korrektheit, Regelkonformität, Nachvollziehbarkeit der Begründung, technischer Verwendbarkeit und Risiko von Folgefehlern. Die Bewertung erfolgt ohne Kenntnis darüber, welche Architekturvariante die jeweilige Ausgabe erzeugt hat. Alle Reviews, Austauschrunden und Expertengespräche werden protokolliert. Diese Protokolle dienen als qualitative Datenquelle und ermöglichen es, nicht nur numerische Bewertungen zu erfassen, sondern auch Begründungen, wiederkehrende Kritikpunkte und Hinweise zur praktischen Verwendbarkeit auszuwerten.

Auf der nutzerseitigen Ebene werden Nutzertests mit mindestens fünf Teilnehmenden sowohl aus dem Smart-Planning-Projektcontext als auch außerhalb des Projektteams durchgeführt. SUS liefert einen quantifizierbaren Score zur wahrgenommenen Gebrauchstauglichkeit, UEQ differenziertere Aussagen über wahrgenommene Nachvollziehbarkeit, Effizienz und Verlässlichkeit. RAGAS wird ergänzend für jene Teilaspekte eingesetzt, bei denen textbasierte RAG-Metriken aussagekräftige Hinweise liefern können – für JSON-Korrekturvorschläge, Tool-Ausgaben und operative Systementscheidungen reichen klassische RAG-Metriken jedoch nicht aus und werden durch technische sowie fachliche Bewertung ersetzt.

5.4 Entscheidungslogik für den Architekturvergleich

Am Ende der Evaluation werden die Ergebnisse beider Systemvarianten systematisch gegenübergestellt. Die graph-basierte Systemarchitektur gilt dann als vorteilhaft, wenn sie im Vergleich zur monolithischen Systemprompt-Struktur weniger fachlich falsche oder unbelegte Korrekturen erzeugt, weniger Folgefehler verursacht, stabilere Ergebnisse bei wiederholten oder variierten Eingaben liefert und von Expertinnen und Experten als nachvollziehbarer bewertet wird.

Es ist dabei möglich, dass die graph-basierte Variante nicht in allen Dimensionen gleichermaßen besser abschneidet – sie könnte etwa die Nachvollziehbarkeit deutlich verbessern, gleichzeitig aber mehr technische Komplexität verursachen, oder vor allem bei komplexen Fehlerfällen Vorteile bieten, während bei einfachen Fehlern kein relevanter Unterschied besteht. Diese differenzierte Betrachtung ist für die wissenschaftliche Aussagekraft der Arbeit zentral. Die Evaluation verfolgt kein vorab festgelegtes Ergebnis, sondern soll eine belastbare Grundlage schaffen, auf der nachvollziehbar beantwortet werden kann, ob die graph-basierte Systemarchitektur im untersuchten Anwendungsfall besser, gleichwertig oder nur unter bestimmten Bedingungen vorteilhaft ist.

6 Erwartete Ergebnisse

Ausgehend von der bisherigen praktischen Erfahrung mit dem bestehenden System wird erwartet, dass die graph-basierte Systemarchitektur insbesondere bei komplexen Validierungs- und Korrekturaufgaben Vorteile gegenüber der monolithischen Systemprompt-Struktur zeigen kann. Diese Erwartung basiert auf der Annahme, dass eine explizite Zerlegung des Entscheidungsprozesses in mehrere klar definierte Verarbeitungsschritte die Kontrolle, Nachvollziehbarkeit und Prüfbarkeit der Systemausgaben verbessert. Gleichzeitig wird nicht davon ausgegangen, dass die graph-basierte Variante in allen Fällen automatisch überlegen ist – die Arbeit soll gerade herausarbeiten, in welchen Situationen ein messbarer Vorteil entsteht und wo mögliche Grenzen dieses Ansatzes liegen.

Hinsichtlich der Halluzinationsrate wird erwartet, dass die graph-basierte Systemarchitektur bei komplexen Snapshots mit mehreren Abhängigkeiten, unvollständigen Referenzen oder mehrdeutigen Fehlerbildern weniger fachlich falsche oder nicht belegbare Korrekturvorschläge erzeugt. Der Grund liegt darin, dass einzelne Teilaufgaben – Fehlerklassifikation, Regelermittlung, Korrekturgenerierung und technische Prüfung – getrennt voneinander betrachtet und validiert werden können. Dadurch soll verhindert werden, dass das Modell in einem einzigen, umfangreichen Prompt-Kontext zu viele Anforderungen gleichzeitig verarbeiten muss und dabei relevante Constraints vernachlässigt – ein Muster, das (Ji et al., 2023) als charakteristisch für LLM-Systeme mit hoher Kontextlast beschreiben. Bei einfachen, gut strukturierten Testfällen mit eindeutiger Regelzuordnung ist hingegen kein relevanter Unterschied zwischen beiden Varianten zu erwarten.

Im Bereich Nachvollziehbarkeit wird ein deutlicherer Vorteil der graph-basierten Variante erwartet. Während beim monolithischen Systemprompt viele Verarbeitungsschritte innerhalb eines einzigen Prompt-Dokuments verschmelzen, soll die graph-basierte Systemarchitektur

explizite Zwischenzustände erzeugen – wodurch sichtbar wird, welcher Verarbeitungsschritt zu welcher Entscheidung geführt hat. (Besta et al., 2024) zeigen, dass genau diese Sichtbarmachung von Reasoning-Schritten ein zentrales Merkmal graph-basierter Architekturen ist. Erwartet wird somit, dass Fehlerursachen dadurch gezielter lokalisiert werden können und sowohl Entwicklerinnen und Entwickler als auch Fachpersonen besser nachvollziehen können, warum eine bestimmte Korrektur vorgeschlagen wurde.

Bei der Robustheit wird eine moderatere Verbesserung erwartet. Das bedeutet nicht, dass die stochastische Natur des LLMs eliminiert wird – erwartet wird vielmehr, dass der Entscheidungsraum pro Verarbeitungsschritt stärker eingegrenzt wird und das System dadurch konsistentere fachliche Entscheidungen trifft. Besonders relevant ist dies bei wiederholten Testläufen, ähnlichen Fehlerfällen und iterativen Korrekturschleifen. Gleichzeitig ist es möglich, dass die graph-basierte Architektur in bestimmten Grenzfällen selbst neue Schwächen zeigt – etwa wenn ein Verarbeitungsknoten mit unerwarteten Eingaben konfrontiert wird und der Fehler nicht kontrolliert weitergegeben wird. Solche Fälle wären wichtige Hinweise für die Weiterentwicklung der Architektur.

Aus praktischer Sicht wird erwartet, dass die Arbeit konkrete Hinweise für die Weiterentwicklung des bestehenden Systems liefert. Sollte sich zeigen, dass die graph-basierte Systemarchitektur insbesondere bei komplexen Fehlerfällen bessere Ergebnisse erzielt, kann daraus eine Empfehlung für die schrittweise Weiterentwicklung des Systems abgeleitet werden. Falls die Vorteile nur in bestimmten Dimensionen auftreten – etwa bei der Nachvollziehbarkeit, aber nicht bei der Halluzinationsrate – wäre auch dieses Ergebnis relevant, da es eine differenzierte Entscheidungsgrundlage für die weitere Systemgestaltung bietet.

Zusammenfassend wird erwartet, dass die Arbeit drei zentrale Ergebnisse liefert – Einen Vergleich zwischen monolithischer Systemprompt-Struktur und graph-basierter Systemarchitektur, einen methodischen Beitrag zur Bewertung strukturierter LLM-Ausgaben im industriellen Kontext und praxisnahe Gestaltungsempfehlungen für den Einsatz von LLM-Systemen in produktionskritischen Validierungs- und Korrekturprozessen. Die Ergebnisse sollen damit sowohl für das konkrete Praxisprojekt als auch für vergleichbare industrielle KI-Anwendungen nutzbar sein.

7 Relevanz für den Studiengang Smart Engineering

Die Bachelorarbeit ist für den Studiengang Smart Engineering relevant, da sie industrielle Produktionsprozesse, datengetriebene Systementwicklung und den Einsatz künstlicher Intelligenz miteinander verbindet. Im Mittelpunkt steht ein reales LLM-gestütztes System, das strukturierte Produktionsplanungsdaten verarbeitet und bei der Validierung sowie Korrektur von JSON-Snapshots unterstützt.

Der Bezug zu Smart Engineering ergibt sich vor allem durch die Verbindung von technischer Systemarchitektur und praktischer industrieller Anwendung.

Die Arbeit untersucht nicht nur, ob ein Large Language Model plausible Antworten erzeugen kann, sondern wie ein KI-System so gestaltet und bewertet werden muss, dass es in einem produktionsnahen Umfeld nachvollziehbare, robuste und technisch verwertbare Ergebnisse liefert.

Besonders relevant ist dabei die Frage der Messbarkeit von Qualität. In industriellen Anwendungen reicht es nicht aus, dass KI-Ausgaben sprachlich überzeugend wirken. Sie müssen fachlich korrekt, technisch ausführbar und im Zielsystem überprüfbar sein. Die Arbeit leistet daher einen Beitrag zur Bewertung strukturierter LLM-Ausgaben und zur Gestaltung verlässlicher KI-Systeme in produktionskritischen Prozessen.

Darüber hinaus zeigt die Arbeit, wie praktische Erfahrungen aus einem Industrieprojekt wissenschaftlich aufgearbeitet werden können. Die im Projekt beobachteten Herausforderungen – etwa Halluzinationen, fehlende Nachvollziehbarkeit und instabiles Verhalten bei variierenden Eingaben – werden in eine konkrete Forschungsfrage überführt und methodisch untersucht.

Damit verbindet die Arbeit zentrale Themen des Smart Engineering. Produktionsplanung, Datenqualität, KI-gestützte Automatisierung, Systemarchitektur und empirische Evaluation in einem realen industriellen Anwendungskontext.

8 Zeitplan

Die finale Abgabe der Bachelorarbeit ist für den 15. September vorgesehen. Da bereits am 15. August eine weitgehend vollständige Fassung an den Betreuer übermittelt werden soll, liegt die zentrale Arbeitsphase zwischen dem 15. Juni und dem 15. August. Bis dahin sollen die theoretischen Grundlagen, die Konzeption und Implementierung der graph-basierten Systemarchitektur, die Evaluation sowie die schriftliche Ausarbeitung weitgehend abgeschlossen sein. Der Zeitraum danach dient vor allem der Einarbeitung von Feedback, der formalen Überarbeitung und der finalen Qualitätssicherung.

Da die praktische Umsetzung agil erfolgt, wird der Zeitplan in iterativen Phasen geplant. Die Phasen bauen zwar logisch aufeinander auf, überschneiden sich jedoch teilweise. Insbesondere Literaturarbeit, Implementierung, Testfallentwicklung, Evaluation und Verschriftlichung werden parallel weitergeführt und anhand laufender Erkenntnisse angepasst.

Phase 1 – Theoretische Grundlagen und Konzeption

14.06.–30.06.

In der ersten Phase werden die theoretischen Grundlagen der Arbeit vertieft und die Konzeption der graph-basierten Systemarchitektur ausgearbeitet. Dazu zählen die Aufarbeitung der Literatur zu Halluzinationen in LLMs, Graph-of-Thoughts, graph-basiertem Reasoning sowie zur Evaluation von LLM-Systemen. Parallel werden die bestehende

monolithische Systemprompt-Struktur analysiert, erste Verarbeitungsknoten der graph-basierten Variante definiert. Am Ende dieser Phase liegt eine dokumentierte Architekturkonzeption vor, die als Grundlage für die prototypische Umsetzung dient.

Phase 2 – Prototypische Implementierung

01.07.–21.07.

In der zweiten Phase wird die graph-basierte Systemarchitektur prototypisch implementiert. Die Umsetzung erfolgt iterativ, beginnend mit den zentralen Verarbeitungsknoten des Graphen und anschließend erweitert um Kontrollflüsse, Fehlerbehandlung, Zwischenergebnisse und die Integration in das bestehende Vier-Agenten-System. Erste technische Tests mit ausgewählten Testfällen begleiten die Implementierung, um frühzeitig Anpassungsbedarf zu erkennen und die Architektur schrittweise zu stabilisieren.

Phase 3 – Evaluation und Datenerhebung

15.07.–04.08.

In der dritten Phase werden beide Systemvarianten systematisch mit dem Testfallkatalog ausgeführt. Die erzeugten Outputs, JSON-Korrekturvorschläge, Validierungsergebnisse, Trace-Logs und Zwischenergebnisse werden dokumentiert und vergleichbar aufbereitet. Zusätzlich werden Expertenreviews durchgeführt und protokolliert. Ergänzende Nutzertests können in dieser Phase vorbereitet und durchgeführt werden, sofern sie für die Bewertung der wahrgenommenen Nachvollziehbarkeit und Verlässlichkeit sinnvoll einsetzbar sind.

Phase 4 – Auswertung und Verschriftlichung

29.07.–15.08.

Die vierte Phase umfasst die Auswertung der Evaluationsergebnisse und die vollständige Verschriftlichung der Arbeit. Die Ergebnisse werden entlang der drei Qualitätsdimensionen Halluzinationsrate, Nachvollziehbarkeit und Robustheit analysiert. Zusätzlich werden die technischen Validierungsergebnisse und die protokollierten Experteneinschätzungen ausgewertet. Das Schreiben beginnt nicht erst nach Abschluss der Evaluation, sondern läuft parallel zu den vorherigen Phasen. Spätestens am 15. August liegt eine weitgehend vollständige, korrekturlesbare Fassung vor, die an den Betreuer übermittelt wird.

Phase 5 – Feedback und Finalisierung

16.08.–15.09.

Nach der Übermittlung der ersten vollständigen Fassung folgt die finale Überarbeitungsphase. In diesem Zeitraum werden Rückmeldungen des Betreuers eingearbeitet, einzelne Kapitel nachgeschärft, formale Anforderungen geprüft und Abbildungen, Quellenverzeichnis sowie Anhänge finalisiert. Der Schwerpunkt dieser Phase liegt nicht mehr auf grundlegender Implementierung oder neuer Evaluation, sondern auf Qualitätssicherung, sprachlicher Überarbeitung und finaler Abgabevorbereitung. Die finale Einreichung der Bachelorarbeit erfolgt am 15. September.

9 Benötigtes Equipment für den praktischen Teil der Arbeit

Für die Durchführung der Bachelorarbeit wird auf das bestehende LLM-gestützte Multi-Agenten-System aus dem Praxisprojekt zurückgegriffen. Dieses bildet die technische Ausgangsbasis für den Vergleich zwischen der bestehenden monolithischen Systemprompt-Struktur und der neu zu konzipierenden graph-basierten Systemarchitektur.

Benötigt wird außerdem der Zugriff auf die bestehende Azure-Umgebung, insbesondere Azure OpenAI, Azure AI Foundry, Azure AI Search, Storage-Komponenten sowie die vorhandene Entwicklungs- und Deployment-Umgebung. Für Implementierung, Tests und Auswertung werden zusätzlich Entwicklungswerkzeuge wie Python, Git, Visual Studio Code sowie Logging- und Monitoring-Möglichkeiten verwendet.

Als Datengrundlage dienen anonymisierte bzw. freigegebene ERP- und Smart-Planning-Snapshot-Daten. Diese werden für die Erstellung repräsentativer Testfälle genutzt, etwa für fehlende Pflichtfelder, ungültige Referenzen, inkonsistente Datenbeziehungen oder unvollständige JSON-Strukturen. Ergänzend werden technische Validierungsergebnisse, Trace-Logs und strukturierte Systemausgaben beider Architekturvarianten dokumentiert.

Für die fachliche Bewertung werden Expertinnen und Experten aus dem Projekt-, Kunden- oder AI-Umfeld einbezogen. Ihre Einschätzungen zu Korrektheit, Nachvollziehbarkeit und praktischer Verwendbarkeit der Systemausgaben werden über Bewertungsraster sowie protokollierte Reviews erfasst und als qualitative Datenquelle in die Arbeit einbezogen.

10 Literatur

- Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Podstawski, M., Gianinazzi, L., Gajda, J., Lehmann, T., Niewiadomski, H., Nyczyk, P., & Hoefler, T. (2024). Graph of Thoughts: Solving Elaborate Problems with Large Language Models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16), 17682–17690.
<https://doi.org/10.1609/aaai.v38i16.29720>
- Es, S., James, J., Espinosa Anke, L., & Schockaert, S. (2024). RAGAs: Automated Evaluation of Retrieval Augmented Generation. In N. Aletras & O. De Clercq (Hrsg.), *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations* (S. 150–158). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.eacl-demo.16>
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12), 1–38. <https://doi.org/10.1145/3571730>
- Wen, Y., Wang, Z., & Sun, J. (2024). MindMap: Knowledge Graph Prompting Sparks Graph of Thoughts in Large Language Models. In L.-W. Ku, A. Martins, & V. Srikumar (Hrsg.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (S. 10370–10388). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.acl-long.558>

11 Anmerkungen

Keine weiteren Bemerkungen.